

PoC of a micro-agent prototype

Marco Mascheroni, Cursor AI Agent :)
University of California San Diego

CMS O&C Week

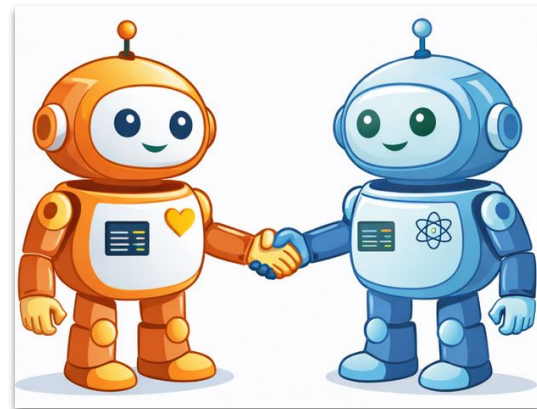
A Technical Proof of Concept

- This talk focuses on the implementation of a work-unit orchestrator prototype
- The afternoon talk will discuss how this model fits the broader architectural strategies
- Thanks to Brian, Miron, Dima, Hassan, Hasan, and Gregor for discussions and inspiration
 - Also thanks to AI coding agents, they made rapid prototyping possible



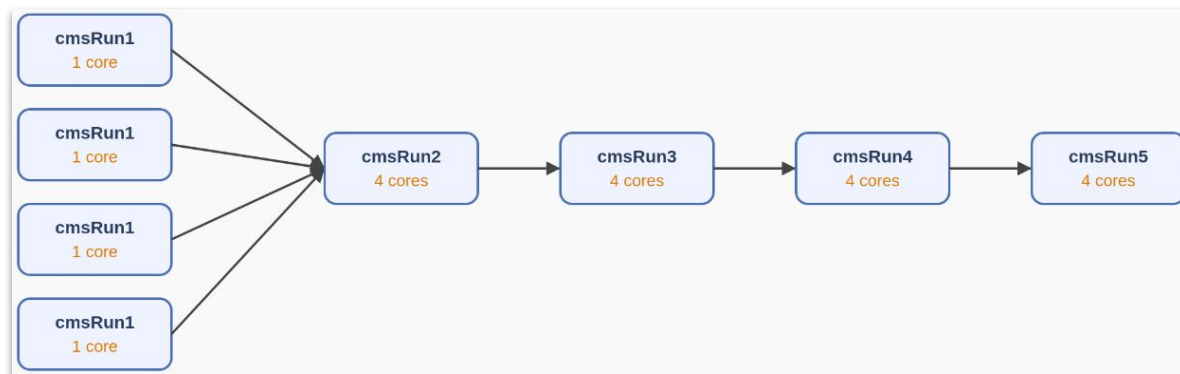
Two Proofs of Concept

- We intentionally explored **multiple implementation strategies in parallel**
- **Dima's prototype**
 - DAGMan-based orchestration
 - Iterative parameter estimation (memory, walltime, CPUs)
 - Physics validation
- **This prototype**
 - Focuses on operational maintainability and scalability testing
 - Minimal execution model built directly on HTCondor
- The goal is exploration, not competition



Where the idea came from

- Discussions around the CMS-led workflow solution
- Collaboration with Andrea, Brian, and Miron
- Conversations with Dima about spec-driven development:
 - **Multiple generators can run in parallel within the same job**
 - We should not be limited to a single cmsRun per step, that constraint came from the pre-multicore WMRuntime design.



Evolution of the Prototype (Feb → Mar 2026)



v0.1.0 — Initial prototype

- Event-based StepChain splitter
- Worker executes stepchain from request.json

v0.1.1 — Grid validation

- Fixes after running on the Grid

v0.2.0 — Full execution loop

- Local stageout via WMCore StageOutMgr
- End-to-end execution on HTCondor

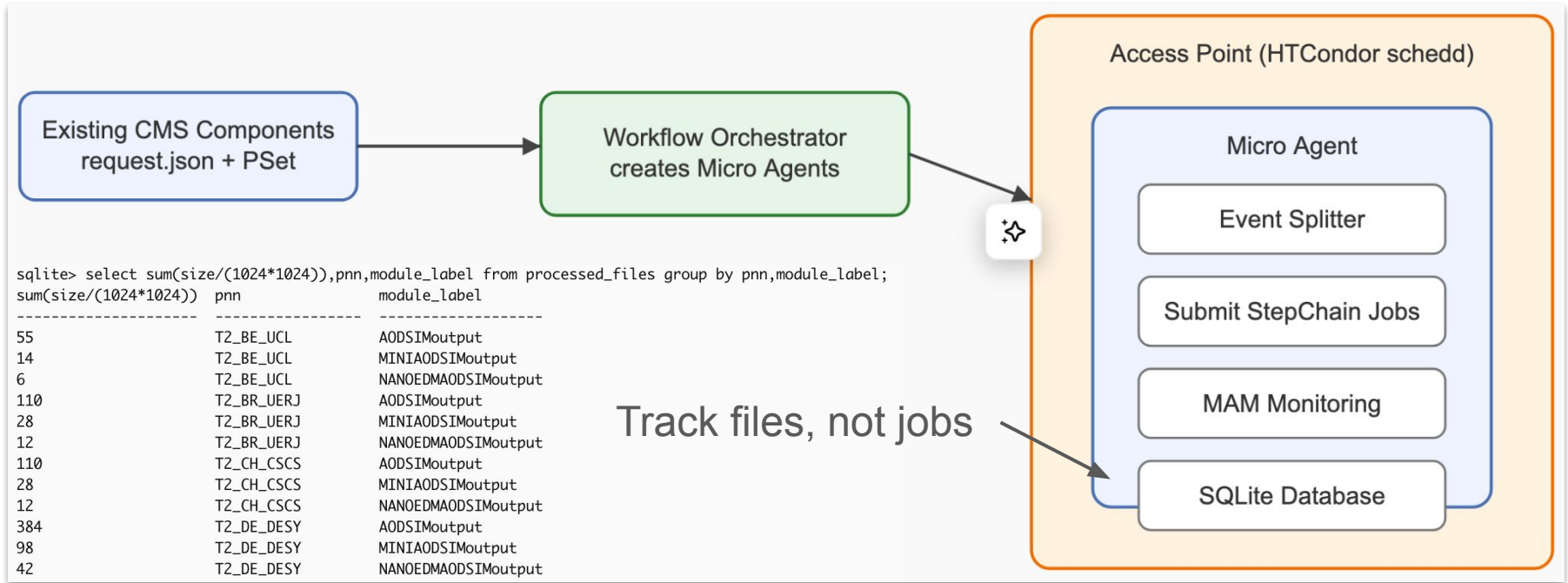
v0.3.0 — Parallel generators

- **NumCopies** support for step 1
- Parallel GEN-SIM execution

v0.4.0 / v0.5.0 — Observability & orchestration

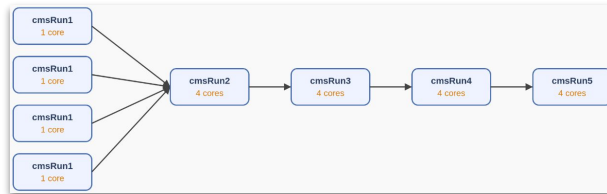
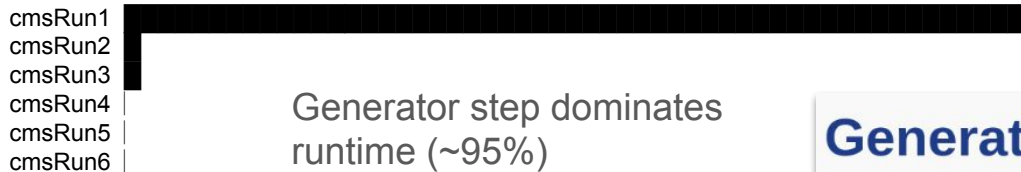
- Micro Agent Monitor (MAM)
- prmon integration
- Workflow orchestrator + Go micro-agent

Work-Unit Orchestrator Prototype Architecture



Impact of NumCopies on Generator Step

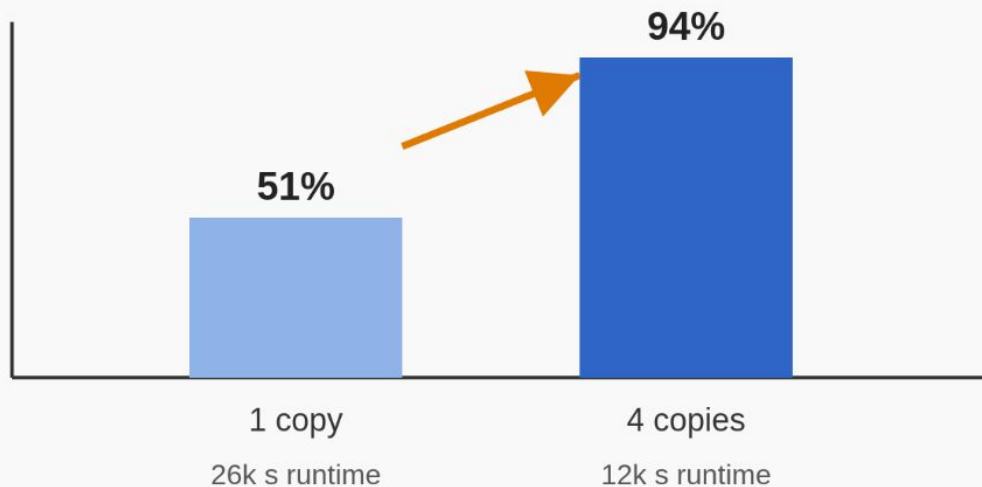
Runtime per step (1 copy job)



- Took a generator dominated workflow:
WplusJetsToTauNu_TauToMuorE_H2
ErratumFix_PDFExt_TuneCP5_5020GeV
- cmsRun1 = 26,453s, others < 1,000s
- Running multiple generator copies in parallel improves CPU utilization:
 - CPU efficiency increases from 51% → 94%
 - Walltime of the dominant step is almost halved

Generator Step Efficiency (cmsRun1)

Impact of NumCopies

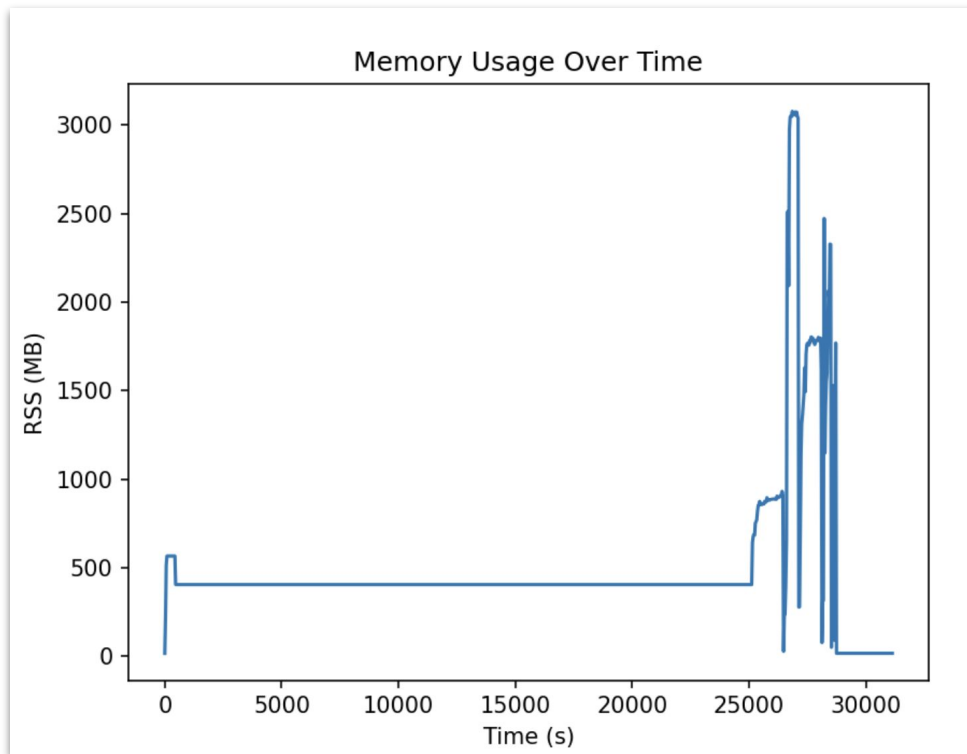


Parallel generator copies dramatically improve CPU utilization

For details click [here](#)

prmon integration

- **prmon monitors resource usage of running jobs**
 - CPU, memory, I/O over time
- **Runs alongside the job as a lightweight process**
- Produces structured output:
 - prmon.txt (human-readable)
 - prmon.json (machine-readable)
- Enables **detailed profiling of workflow steps**
 - identify memory peaks
 - track CPU efficiency
- Data is returned with job output and can be **aggregated centrally**



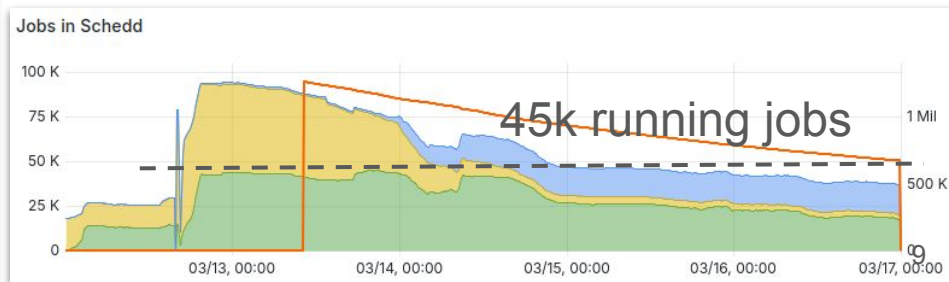
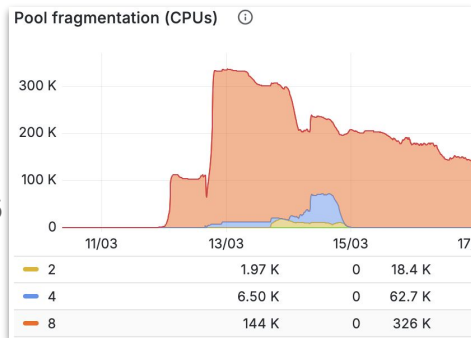
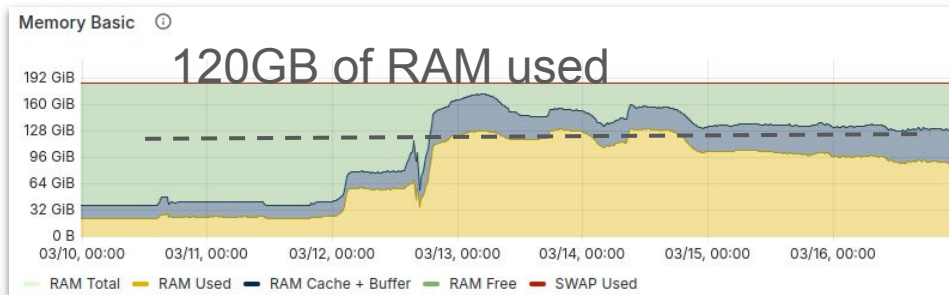
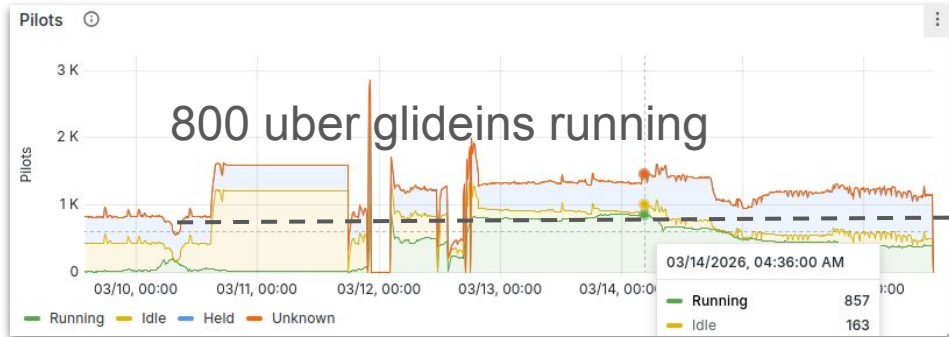
Prototype Scale Tests

- Scheduler **memory constraint** is the main concern
 - ~2 MB per running job
 - Micro-agent: ~30 to ~50 in python, ~3 to ~5 in Go
 - ~ 10× lower overhead with Go
- **Current capacity 2TB** across production schedds
 - 500k running jobs (2M cores with 4 cores) → ~1 TB RAM
 - 200k → 1 TB available is divided by ~5 MB per micro agent
 - Practical target: ~100k concurrent micro agents

~ 45k running
jobs (300K cores)

~ 1 million idle
unmaterialized jobs

duty cycle 0.3



Conclusion

Architectural validation

- Work-unit orchestration integrates naturally with HTCondor
- Micro-agent execution model works in practice

Performance insights

- Parallel generator copies significantly improve CPU efficiency

Operational considerations

- Lightweight orchestration layer can fulfill HL-LHC needs through horizontal scalability
- Go implementation keeps the micro-agent footprint small

Experimentation value

- Rapid prototyping enabled extensive experimentation
- Explored new tooling, including AI coding assistants

Prototype available at: github.com/mmascher/WorkflowOrchestrator

More on how this integrates with other systems in my afternoon talk



(Just for fun)